Liquor Counter POS

Knowledge Transfer & Handoff

A point-of-sale, inventory and accounting platform for liquor retail stores and bars in India, for single stores or multi-store syndicates.

Prepared	7 November 2026
Audiences	Part A: customer / stakeholders (sections 1). Part B: development team (sections 2-4).
Source of truth	AGENTS.md (project context), PROGRESS.md (history), INTEGRATIONS.md (external setup), this document (summary).
State of the code	All work described here is in the working tree and **not committed**. The repository owner decides when to commit.

1. High-level & customer view

Core value proposition

Liquor retail in India is heavily regulated and cash-heavy. Stores lose money to stock leakage, mistaken prices, unrecorded cash, and slow, error-prone month-end accounting. This platform puts the whole shop on one system: every bottle sold or received is recorded, prices come from the database (never typed at the counter), tax is calculated correctly, each cashier's cash is counted against their shift, and the books are produced automatically and can be exported to Tally.

Primary users: store owners and syndicate (multi-outlet) owners, branch managers, cashiers and bar stewards. A separate platform Super Admin runs the SaaS side (tenants, plans, billing).

Role	What they do
Platform Super Admin	Manages tenants/syndicates, plans and billing, integrations status, e-mail outbox.
Owner (syndicate)	Sees all outlets, consolidated invoices, bank statements for syndicate accounts, branch picker.
Manager	Voids bills, closes the day, approves supplier invoices, accounting, schemes, compliance, bank reconciliation.
Cashier	Bills at the counter (POS), opens/closes own shift, khata (credit) customers, counter sheets.
Steward	Bar table service: tables and kitchen/bar order tickets (KOT) only.

Key features and capabilities

- **POS billing** - barcode/scanner billing; MRP is tax-inclusive; GST back-calculated per line after discount; payment by Cash, Card, UPI or Credit; offline queue that syncs when the connection returns; automatic receipt printing via a local print agent.
- **Bills** - history and search; bills are never deleted, only voided (manager, reason required) with stock and credit reversed.
- **Shifts and registers** - terminals, cashier shifts with denomination count and discrepancy.
- **Counter sheets** - daily Opening + Inward - Closing = Sold sheets for stores that bill in bulk (exportable).
- **Bar service** - tables, sessions, KOT for kitchen and bar, table bill; kitchen and bar menus; pour control.
- **Inventory and purchasing** - stock per branch with a full audit trail; supplier invoice upload (CSV/XLSX/PDF/OCR), review and approve; indent suggestions.
- **Khata (customer credit)** - credit limits, repayments, statements, outstanding report.
- **Schemes and compliance** - supplier scheme slabs and payouts; excise quotas, lifting report, breakage log, stock-audit report.
- **Accounting** - automatic double-entry vouchers from sales, purchases, expenses and repayments; manual journals; trial balance, P&L, balance sheet; Tally XML export.
- **Day closing** - end-of-day cash/card/UPI/credit split with discrepancy.
- **Bank statement reconciliation (new)** - upload any bank's statement (PDF/CSV/XLSX), validate it with a running-balance check, match lines to the books, post missing entries, and ignore personal/non-store transactions with a required reason and saved rules.
- **Role-based dashboards and new login (new)** - attention strip, same-time-yesterday comparison, hourly sales, payment mix, closing checklist, khata ageing; login with lockout countdown and server status.
- **SaaS administration** - tenants, plans, Razorpay billing, e-mail reminders, user and page permissions, syndicates and consolidated invoices.

Current project status

Area	Status
Phases 1-6 (tenancy, shifts, counter sheets, khata, schemes, compliance, accounting, React UI, supplier invoices)	Done and verified (see PROGRESS.md).

Area	Status
Security / hardening pass (login throttle, production start-up guards, headers, closing guards)	Done and tested.
Bank reconciliation incl. ignore rules	Built and tested on synthetic statements only. **Not yet tested on real bank statements** - the customer will supply them; expect parser tweaks.
Dashboard and login redesign	Done; manager dashboard checked live. Cashier and owner views and phone width are covered by automated tests but were not all eyeballed.
Verification (7 Nov 2026)	TypeScript clean (backend and frontend); Vitest 728/728 (57 files); npm audit 0 vulnerabilities; migrations in sync; Playwright 42 of 43 pass (only the legacy counter-sheet spec fails).
Source control	Everything above is uncommitted. Suggested commits: earlier fixes, bank module (with ignore rules), dashboard/login.
Planned / open	Real-statement parser tuning; roll the new look out to the sidebar and other pages; Redis/BullMQ background jobs (listed as 'Phase 2' in .env.example, not used today); production hosting and CI/CD (none defined in the repo).

Known issues and technical debt

- **No CI/CD or container definition in the repo.** No Dockerfile, compose file or pipeline was found. Deployment must be defined by whoever hosts it.
- **Legacy UI.** preview.html is the original single-file UI (served at /preview) kept as a backup. The React app in frontend/ is the real UI. The legacy counter-sheet e2e spec targets preview.html and fails; it is intentionally left alone.
- **Login throttle is in memory.** Correct for one server process; with several instances each keeps its own counters. Move it to Redis/DB before scaling out. Set TRUST_PROXY behind a proxy or all clients look like one IP.
- **Mock payment gateway.** Default PAYMENT_PROVIDER=mock lets a tenant admin mark invoices paid for free; production refuses to start with it unless ALLOW_MOCK_PAYMENTS=1.
- **PGlite vs Postgres.** Tests run on embedded PGlite, which serialises transactions, so row-lock behaviour (FOR UPDATE) is only truly exercised on real PostgreSQL. Load-test concurrency (stock, bill numbers) there.
- **Domain simplifications:** MRP is global, not per branch; purchases use periodic inventory (no automatic closing-stock entry); scheme payouts use the 'best slab reached' rule; closing selects bills by creation date, so voiding a bill from an already-closed day is not reflected in that closing.
- **Mixed visual themes.** Login and dashboards use the new brass/green look (CSS scoped with an lc- prefix); sidebar and other pages are still purple.
- **Lint/format.** Only the frontend has a linter (oxlint). There is no backend lint or formatter configured.
- **Working-tree oddities to check before committing:** bun.lock and pnpm-lock.yaml show as deleted, preview.html and .Jules/palette.md show as modified; confirm these are intended. Two lockfile managers exist (package-lock.json at root, frontend/pnpm-lock.yaml and frontend/package-lock.json) - pick one.
- **Windows note:** PowerShell 5.1 Get-Content mis-displays UTF-8; use -Encoding utf8.

2. Technology stack & architecture

Layer	Choice
Frontend	React 19, TypeScript 5.7, Vite 8, react-router-dom 7, Tailwind CSS 4, lucide-react icons, self-hosted fonts via @fontsource (Bricolage Grotesque, Hanken Grotesk, IBM Plex Mono). **State:** React context providers only (Theme, Auth, Loading) plus local component state; no Redux or query library. Pages call a small fetch wrapper in frontend/src/api/client.ts. Dark mode is a .dark class; new CSS is scoped under .lc with lc- prefixed classes. Offline billing queue is in localStorage.
Backend	Node.js 22+, ESM, TypeScript strict (NodeNext, imports use the .js extension), Fastify 5 with @fastify/jwt, cors, helmet, rate-limit, multipart, static, sensible. Validation with zod 4. Money with decimal.js (never JS floats). Passwords with bcryptjs. **API style:** REST/JSON under /api/v1 (plus /health and /api/v1/health), JWT bearer auth.
Files and parsing	exceljs, papaparse, pdfjs-dist, tesseract.js (OCR) and @napi-rs/canvas for supplier invoices and bank statements; nodemailer for e-mail.
Database	PostgreSQL through Drizzle ORM (postgres-js). For zero-setup dev and all tests, embedded PGlite: DATABASE_URL=pglite://./.pglite or pglite://memory. The whole schema is in src/db/schema.ts; SQL migrations in drizzle/ (currently up to 0011_bank_reconciliation).
Caching / queues	None in use. Redis/BullMQ is only a commented placeholder. A billing job runs inside the server every 6 hours.
File storage	No cloud storage. Uploaded statements are not stored (only hash, parsed lines and mapping); supplier invoice files are kept via the invoices module.
DevOps / infra	**Not defined in this repo** - no cloud provider, Docker, Kubernetes or pipeline files. Production build is npm run build (builds the frontend into dist/frontend, then tsc) and npm start (node dist/index.js with .env). The server serves the built frontend itself.
Testing tools	Vitest 5 (unit + integration on in-memory PGlite), Playwright (e2e against a fresh PGlite DB), supertest.

Third-party integrations

Integration	Detail
Razorpay	Subscription payments (UPI, cards, netbanking). Checkout signature verified server-side plus webhook at /api/v1/webhooks/razorpay; idempotent. Built-in mock gateway by default.
SMTP e-mail	Any SMTP server (password reset, billing reminders). Without SMTP_HOST, e-mails are stored in the email_outbox table and visible to Super Admin.
Local print agent	print-agent/ folder; a small program at the store that prints receipts; token created in Business Settings.
Tally	Accounting export as Tally XML (no live connection).
Auth / analytics	No external auth provider (own JWT + bcrypt). No analytics or monitoring SDK found.

Request flow

```
Browser (React)  --HTTPS-->  Fastify  --routes (auth, zod validation, role/branch checks)
                                      --service (db.transaction, SQL via Drizzle)
                                      --engine  (pure business rules, unit-tested)
                                      --domain events (bill.paid, bill.voided, ...) -> accounting posts
                                        balanced vouchers INSIDE the same transaction
PostgreSQL (or PGlite in dev/tests)
```

3. Codebase & project structure

Directory walkthrough

Path	Purpose
src/app.ts, src/index.ts	buildApp() registers plugins, routes and accounting event handlers; index.ts starts the server (auto-migrates on PGlite). src/dev-local.ts is the zero-setup runner.
src/db/	schema.ts (all tables), client.ts (driver chosen from DATABASE_URL), migrate.ts, seed.ts (demo data), create-super-admin.ts, clear-inventory.ts.
src/lib/	errors.ts (typed errors + sendError), business-day.ts (IST business date), domain-events.ts.
src/middleware/	auth.guard.ts: requireRole, assertBranchAccess, getAccessibleBranchIds.
src/config/	integrations.ts: payments/e-mail config and the production safety check.
src/modules/<domain>/	One folder per domain with .engine.ts (pure logic), .service.ts (DB + zod), .routes.ts (HTTP). Domains: accounting, auth, bank, billing, branches, closing, compliance, counters, customers, dashboard, expenses, inventory, invoices, kot, menu, notifications, payments, platform, printing, products, schemes, shifts, stock, syndicates, tables, terminals, users.
drizzle/	Generated SQL migrations and snapshots (never edit by hand; use db:generate).
frontend/src/	React app: one folder per screen (pos, bills, dashboard, bank, accounting, ...), plus auth/, components/ (Layout, ErrorBoundary), context/, api/, lib/ (auto-print, business-day).
frontend/src/dashboard/	New dashboards: Dashboard.tsx shell, Cashier/Manager/OwnerView, charts, parts, lc.css.
tests/unit, integration, e2e	Engine tests; real-app + real-SQL tests on PGlite via setupIntegration(); Playwright journeys.
print-agent/	Local receipt-printing agent for stores.
scripts/, public/	Utility scripts; static assets.
preview.html	Legacy single-file UI backup, served at /preview. Do not extend.
AGENTS.md, PROGRESS.md, INTEGRATIONS.md, PLAN.md, TECHNICAL_DOCUMENTATION.md	Project docs. Read AGENTS.md first. .agents/ and README.md describe generic AI tooling, not this app.

Design patterns

- **Modular monolith** with an **engine / service / routes** split per domain: pure functions hold business rules and are unit-tested; services own transactions; routes stay thin.
- **Domain events** for cross-module effects. Accounting subscribes and posts vouchers in the emitting transaction, so a posting failure rolls the sale back.
- **Multi-tenancy by scope**: platform owner (all), syndicate owner (their branches), others (own branch). Every branch-scoped route must **await assertBranchAccess**.
- **Idempotency**: clientRef on bills and repayments; line hashes for bank statements; one voucher per source.
- **Append-only audit trails**: stock_audit, price_audit; bills are voided, never deleted.
- **Guarded atomic SQL** for stock (quantity = quantity - n WHERE quantity >= n); bill numbers from bill_counters upsert.

Configuration files

File	Contents
.env (from .env.example)	DATABASE_URL, JWT_SECRET (32+ chars), JWT_EXPIRES_IN, CORS_ORIGIN (required in production, not *), TRUST_PROXY, PORT, NODE_ENV, APP_URL, PAYMENT_PROVIDER and Razorpay keys, ALLOW_MOCK_PAYMENTS, SMTP_*, MAIL_FROM, BILLING_JOB_ENABLED, BUSINESS_UTC_OFFSET_MINUTES (default 330).
package.json / frontend/package.json	Scripts and dependencies (backend / frontend).
tsconfig.json, frontend tsconfig	TypeScript settings.
drizzle.config.ts	Drizzle Kit config.
vitest.config.ts, playwright.config.ts	Test runners. Playwright boots its own server and DB in ./.pglite-e2e and serves the built dist/frontend.
frontend vite config	Dev server on 5173 proxying /api to the backend on 3001.

4. Developer onboarding & KT

Local setup

Requirements: Node.js 22 or newer and npm. No database install is needed for the zero-setup path.

```
git clone <repository-url> liquor-POS
cd liquor-POS
npm install
cd frontend && npm install && cd ..

# Zero setup (embedded PGlite, data in ./.pglite, seeds demo data on first run):
npm run dev:local              # API + built UI on http://localhost:3001
npm run dev:frontend           # optional: Vite dev UI on http://localhost:5173

# Against real PostgreSQL:
copy .env.example .env         # set DATABASE_URL, JWT_SECRET (32+ chars)
npm run db:migrate
npm run db:seed
npm run dev                    # tsx watch, reads .env
npm run super-admin            # create a platform Super Admin
```

Seed logins (password **Password@123**): owner@liquor.com (syndicate owner), manager@liquor.com, cashier@liquor.com, steward@liquor.com. The login page shows demo accounts in development only.

Testing and linting

```
npm test                        # all Vitest (unit + integration)
npm run test:unit
npm run test:integration
npx tsc --noEmit                # backend type-check
cd frontend && npx tsc -b       # frontend type-check
cd frontend && npm run lint     # oxlint (frontend only)
npm run build:frontend          # REQUIRED before e2e (Playwright serves dist/frontend)
npm run test:e2e                # first time: npx playwright install chromium
npm audit --omit=dev
```

Schema change: edit src/db/schema.ts, then run npm run db:generate (any DATABASE_URL works, for example postgresql://u:p@localhost:5432/x), and commit the new files in drizzle/. Run it again afterwards; it must say 'No schema changes'.

Local frontend testing against the CORS-disabled setup: either use the Vite dev server (proxies /api) or the built UI on port 3001.

Deployment workflow

- **Not defined in the repository.** There is no CI pipeline, container image, staging environment or documented branching strategy. A git repository exists; conventional commits are the stated style (feat(scope): ..., test(e2e): ...).
- **Build and run (any host):** set production .env, run npm ci (root and frontend), npm run build, npm run db:migrate, then npm start. Run behind an HTTPS reverse proxy.
- **Production start-up guards:** the server refuses to start with a short JWT_SECRET, CORS_ORIGIN of *, or the mock payment gateway (unless ALLOW_MOCK_PAYMENTS=1 with MOCK_PAYMENT_SECRET).
- **Recommended minimum pipeline:** on every PR run backend and frontend tsc, npm test, build:frontend and test:e2e, then npm audit; deploy a tagged build to staging, run migrations, smoke-test, promote.
- **Commit rule for this workspace:** never commit or push without explicit approval; no AI attribution in messages.

Critical modules - handle with extra care

Location	Why it is sensitive
src/modules/billing/ (engine + service)	Price, GST-after-discount allocation, stock decrement, bill numbers, void reversal. Errors change money and stock.
src/modules/accounting/posting.service.ts, chart.ts	Subscribes to domain events and posts balanced vouchers inside the sale transaction. A wrong mapping silently corrupts the books.

Location	Why it is sensitive
src/modules/bank/ (bank.engine.ts, service)	Statement parsing, balance-chain validation, matching, ignore rules, and posting vouchers (sourceType BANK_LINE). Never auto-ignore a book-matched line; a posted line cannot be un-matched.
src/middleware/auth.guard.ts, modules/auth/	RBAC, tenancy, login throttle. A missing await on assertBranchAccess silently disables the check.
src/modules/closing/, shifts/, counters/	Cash discrepancy, day close, counter-sheet posting (Sold = Opening + Inward - Closing).
src/modules/inventory/, invoices/	Stock inward, supplier invoice approval (atomic), OCR/CSV parsing.
src/modules/payments/, platform/	Razorpay signature and webhook verification, plans and billing job (PLAN_PRICES in platform.service.ts).
src/modules/dashboard/dashboard.engine.ts	Attention ranking, same-time-yesterday, khata ageing; server-side aggregation (never sum a paged bill list in the browser).
src/db/schema.ts and drizzle/	Single schema file; migrations are generated, never hand-edited.
src/config/integrations.ts	Production safety guard for payments.

First-week checklist for a new developer

- Read AGENTS.md, then PROGRESS.md and INTEGRATIONS.md.
- Run npm run dev:local and log in as each seed role; walk POS, shift, closing, bank and dashboard.
- Run npm test and npx tsc --noEmit; confirm green before changing anything.
- Read one full module end to end (billing is the best example) to learn the engine/service/routes pattern.
- Ask the customer for real bank statements and run them through the bank import preview; fix parser gaps with a new synthetic test per layout.
- Decide hosting, CI and branching with the customer and add the pipeline files to the repo.